ENCE260: Embedded Systems

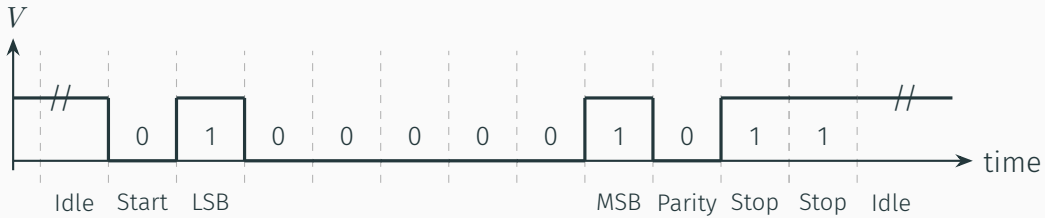
Lecture 8: Makefiles

How to do less typing.

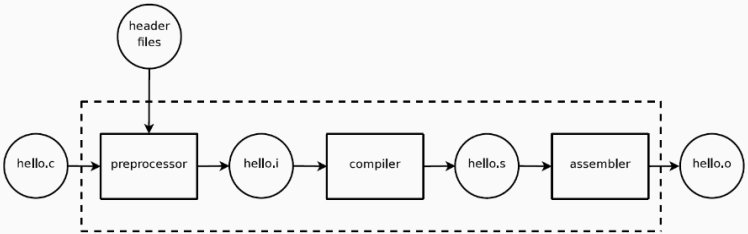
Revision

One UART peripheral is communicating over a wire to another. The UARTs have been configured for 7-bit data, the LSB is transmitted first, and the idle state is high. There is one start bit, two stop bits, and an even parity bit.

Sketch the waveform that should be sent on the wire to transmit the ASCII character `'A'`? Decimal 65, = `0b 100 0001`.



C Compilation Recap



- The preprocessor reads the C source file, the `.h` files, and expands macros.
- The compiler takes the output of the preprocessor, checks for syntax errors, and compiles each function into assembly language.
- The assembler converts the assembly language to machine code, stored in a single binary object file (`.o`) for the “compilation unit”.
- This process happens separately for each `.c` file.

C Compilation Recap

To build a single compilation unit:

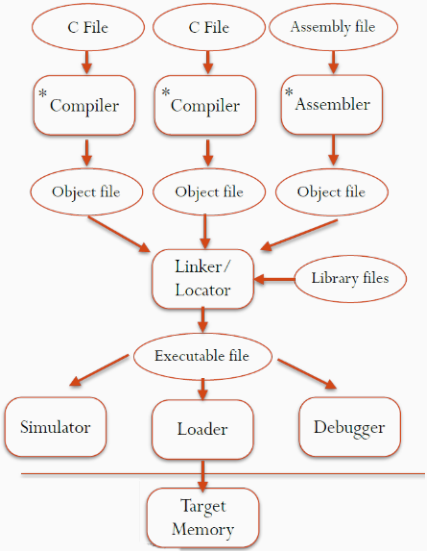
```
gcc hello.c -c -o hello.o
```

To link the object files `hello.o` and `module.o`:

```
gcc hello.o module.o -o hello.out
```

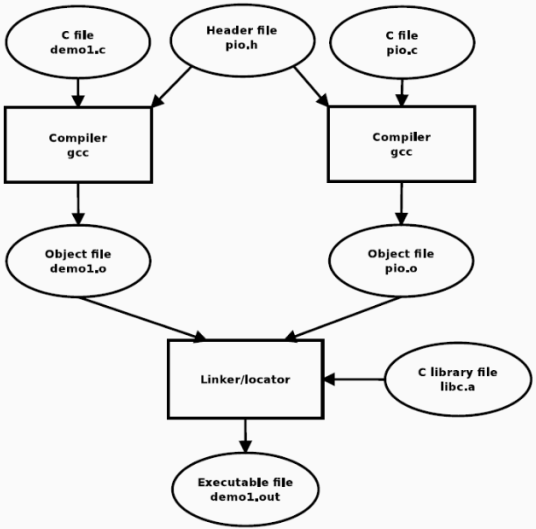
The Linker

- Each C file is compiled independently.
- The linker combines the multiple object files (including library files) into a single executable, e.g. a `.out` file.
- The loader downloads an executable file to the target memory. In the Fun Kit's case it converts it into a `.hex` first.
- For embedded systems, the development PC is the host, and the micro is the target.



Makefiles

- All non-trivial programs are comprised of multiple C files (because of modularity).
- It's painful to type out the commands to build the program each time.
- Instead we put all the commands in a script (makefile).
- This lets us only compile the files whose source has changed.



`make`

- We only need to recompile just the files that have changed.
- The program `make` automates the build process for us (i.e. runs the compiler & linker as needed) to create the executable file.
- `make` reads the `makefile`, which contains a collection of rules.
- These rules specify the dependencies between the files, and the commands to build the executable program.
- Running the command `make` in a terminal will automatically search for a file called `makefile` and then follow it. The name `Makefile` also works.

Example `makefile` for `demo1.c`

```
# Definitions.
CC = avr-gcc
CFLAGS = -mmcu=atmega32u2 -Os -Wall -Wstrict-prototypes -Wextra -g
    -I../../drivers/avr


# Compile: create object files from C source files.
demo1.o: demo1.c ../../drivers/avr/pio.h ../../drivers/avr/system.h
    $(CC) -c $(CFLAGS) $< -o $@

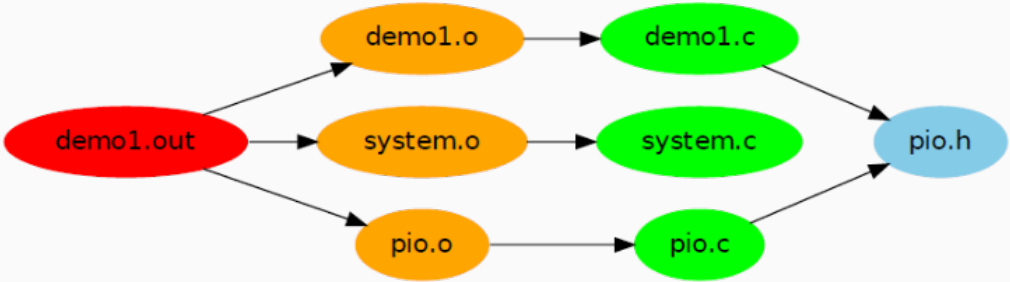
pio.o: ../../drivers/avr/pio.c ../../drivers/avr/pio.h
    ../../drivers/avr/system.h
    $(CC) -c $(CFLAGS) $< -o $@

system.o: ../../drivers/avr/system.c ../../drivers/avr/system.h
    $(CC) -c $(CFLAGS) $< -o $@


# Link: create output file (executable) from object files.
demo1.out: demo1.o pio.o system.o
    $(CC) $(CFLAGS) $^ -o $@ -lm
```

File Dependency Graphs

- File dependency graphs show graphically what files are required to build a program. Can be useful to draw by hand (here I've ignored `system.h`)
- Compilation of `demo1.c` is also conditional on `pio.h`.
- If `demo1.o` already exists, compilation is only performed if `demo1.c` or `pio.h` have been modified since the last compilation.



`makefile` Rule

```
target : prerequisite1 prerequisite2 prerequisite3
	recipieCommandsThat --MakeTarget
	from prerequisites
```

- Can run this rule with:

  `make target`

- The recipe **must** be intended with tab characters; spaces won't do!
- The recipe will only run if any of the prerequisites have changed.
- If the prerequisites are also targets, the dependency graph is followed all the way up.

Variables in `make`

```
1 # Definitions.
2 CC = avr-gcc
3 CFLAGS = -mmcu=atmega32u2 -Os -Wall -Wstrict-prototypes -Wextra -g
      -I../../drivers/avr
4
5
6 # Compile: create object files from C source files.
7 demo1.o: demo1.c ../../drivers/avr/pio.h ../../drivers/avr/system.h
8     $(CC) -c $(CFLAGS) $< -o $@
```

- Here, `CC` is the C compiler we are going to use.
- `CFLAGS` is used to add arguments to the compiler.
- The microcontroller programmed is set by `-mmu=atmega32u2`.
- The location of the drivers is added by `-I../../drivers/avr`.
- We get the value of a variable by using `$(...)`, since otherwise the variable name is treated as a terminal command.

Automatic Variables in `make`

```
1  # Definitions.
2  CC = avr-gcc
3  CFLAGS = -mmcu=atmega32u2 -Os -Wall -Wstrict-prototypes -Wextra -g
       -I../../drivers/avr
4
5
6  demo1.o: demo1.c ../../drivers/avr/pio.h ../../drivers/avr/system.h
7      $(CC) -c $(CFLAGS) $< -o $@
8
9
10 demo1.out: demo1.o pio.o system.o
11     $(CC) $(CFLAGS) $^ -o $@ -lm
```

- `$@` inserts the target of the current rule.
- `$<` inserts the first prerequisite of the current rule.
- `$^` inserts all the prerequisites of the current rule.

`makefile` Phony Targets

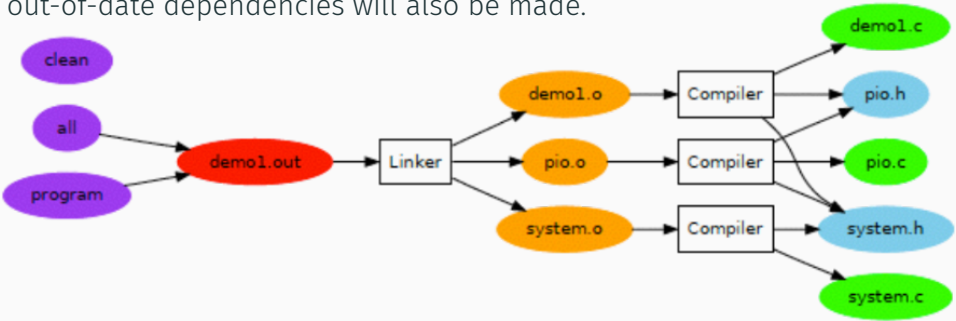
- Some `makefile` targets are not associated with a file.
- These are called phony targets.
- Phony targets often do tasks instead of prepare files; they let `make` do verbs as well as nouns.
- Common phony targets include `clean`, `program`, `install` and `all`.
- E.g., a phony target `clean` and its rule is defined as

```
1 .PHONY: clean
2 clean:
3     rm *.o *.out *.hex
```

- The command can then be run with `make clean`.

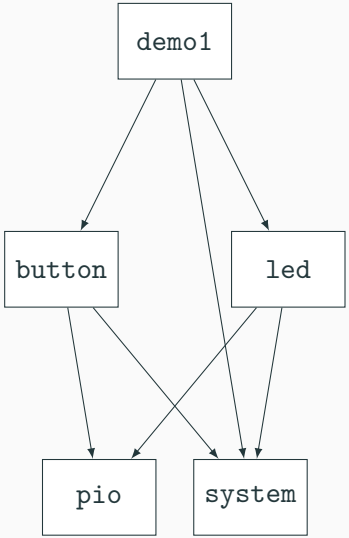
Makefile Dependencies

- For a dependency diagram, arrows show the direction of dependencies. Data flows the other way.
- The command `make` builds the first listed target (often a phony `all`).
- Can specify any target, e.g. `make program` or `make pio.o`, and all "upstream" out-of-date dependencies will also be made.



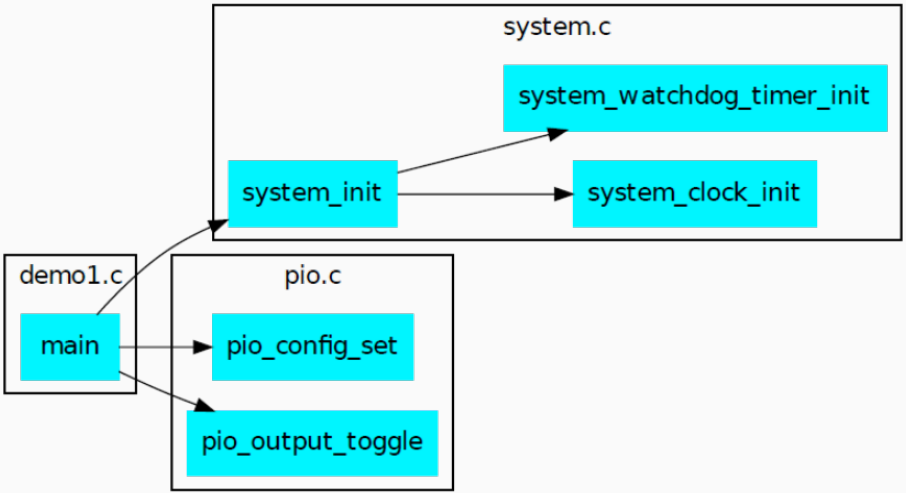
Module Dependencies

- Similar to a makefile dependency diagram, but limited to modules in the code.
- There are usually multiple layers of dependencies.
- Dependencies can skip layers.



Callgraphs

Callgraphs represent the calling relationships between functions in a program.



Assignment Makefile

- In `ence260-ucfk4/assignment/template/` there is a skeleton `makefile` for your assignment.
- It will build the existing `game.c` file in the same directory.
- You can instead base your game's `makefile` on one of the lab ones (e.g. `lab3-ex4`).
- Either way, whenever you add or include additional modules (buttons, LEDs, `ir_uart` etc.), you will need to modify your `makefile` !

Provided Makefile

```
# Definitions.
CC = avr-gcc
CFLAGS = -mmcu=atmega32u2 -Os -Wall -Wstrict-prototypes -Wextra -g -I. -I../../utils -I../../fonts -I../../drivers
    -I../../drivers/avr
OBJCOPY = avr-objcopy
SIZE = avr-size
DEL = rm

# Default target.
all: game.out

# Compile: create object files from C source files.
game.o: game.c ../../drivers/avr/system.h
    $(CC) -c $(CFLAGS) $< -o $@

system.o: ../../drivers/avr/system.c ../../drivers/avr/system.h
    $(CC) -c $(CFLAGS) $< -o $@

# Link: create ELF output file from object files.
game.out: game.o system.o
    $(CC) $(CFLAGS) $^ -o $@ -lm
    $(SIZE) $@

# Target: clean project.
.PHONY: clean
clean:
    -$(DEL) *.o *.out *.hex

# Target: program project.
.PHONY: program
program: game.out
    $(OBJCOPY) -O ihex game.out game.hex
    dfu-programmer atmega32u2 erase; dfu-programmer atmega32u2 flash game.hex; dfu-programmer atmega32u2 start
```

Makefiles Beyond C

For example, this `makefile` runs a python simulation and then plots the results.

```
PLOTS = fig1.png fig2.png

all: $(PLOTS)

data2.csv: runSim.py data1.csv
    python $< data1.csv

fig1.png: plotFig.py data1.csv
    python $< data1.csv

fig2.png: plotFit.py data2.csv
    python $< data2.csv
```

Key Concepts

- When compiling C programs, object files are produced from source files. These object files are then linked to produce an executable.
- `make` reads a `makefile` and allows these steps to run automatically.
- `make` also tracks file dependencies, so that only files downstream of changes get rebuilt. This speeds up large builds.
- For your assignment, you will need to make new `makefile` rules when you add or include new modules.
- `make` can be used to efficiently automate any task run on the command line, not just compilation.

Homework

Consider the template `makefile` for your game, three slides back.

If the file `game.c` is modified and the command `make program` is run, which recipe commands are run, and in what order?